



INSTITUTO POLITECNICO NACIONAL

Escuela Superior de Computo

Desarrollo Móvil

Herencia, Clases Abstractas e Interfaces

Profesora: Monica Rivera De La Rosa

Alumno: Gutiérrez Espinosa Jordan Gabriel

Grupo: 7CM2

Ciclo: 2024-2025(1)



1. Descripción del mecanismo de herencia en Kotlin

La herencia en Kotlin permite que una clase herede propiedades y comportamientos de otra clase. Es uno de los pilares de la Programación Orientada a Objetos (POO). En Kotlin, todas las clases son por defecto **final**, lo que significa que no se pueden heredar a menos que se marquen explícitamente como **open**.

```
open class ClaseBase {  
    // Propiedades y métodos que pueden ser heredados  
}  
  
class ClaseDerivada : ClaseBase() {  
    // ClaseDerivada hereda de ClaseBase  
}
```

Características de la herencia en Kotlin:

- **Clases open:** Una clase debe estar marcada como `open` para que pueda ser heredada.
- **Constructores de clase base:** Si una clase base tiene un constructor primario, las clases derivadas deben llamarlo explícitamente.
- **Métodos y propiedades open:** Al igual que las clases, los métodos y propiedades también deben marcarse como `open` si se desea que puedan ser sobrescritos.
- **Sobreescritura (override):** Cuando una clase derivada sobreescribe una función o propiedad de la clase base, debe usar la palabra clave `override`.

Ejemplo de herencia:

```
// Clase base abierta para ser heredada  
open class Animal(val nombre: String) {  
    open fun sonido() {  
        println("El animal hace un sonido")  
    }  
}  
  
// Clase derivada que hereda de Animal  
class Perro(nombre: String) : Animal(nombre) {  
    override fun sonido() {  
        println("El perro ladra")  
    }  
}  
  
fun main() {  
    val miPerro = Perro("Max")  
    miPerro.sonido() // Salida: El perro ladra  
}
```

Descripción de la creación de clases abstractas e interfaces



Clases abstractas:

Una **clase abstracta** es una clase que no puede ser instanciada directamente, ya que está destinada a ser heredada. Las clases abstractas pueden contener métodos tanto abstractos (sin implementación) como no abstractos (con implementación).

Sintaxis:

```
abstract class ClaseAbstracta {  
    abstract fun metodoAbstracto() // Método sin implementación  
    open fun metodoConcreto() {  
        // Método con implementación  
    }  
}
```

Una clase que herede de una clase abstracta debe proporcionar implementaciones para todos los métodos abstractos, a menos que la clase derivada también sea abstracta.

Ejemplo de clase abstracta:

```
abstract class Vehiculo {  
    abstract fun acelerar() // Método abstracto  
    fun frenar() {  
        println("El vehículo está frenando")  
    }  
}  
  
class Coche : Vehiculo() {  
    override fun acelerar() {  
        println("El coche está acelerando")  
    }  
}  
  
fun main() {  
    val miCoche = Coche()  
    miCoche.acelerar() // Salida: El coche está acelerando  
    miCoche.frenar()   // Salida: El vehículo está frenando  
}
```

Interfaces:

Una **interfaz** define un conjunto de métodos que una clase debe implementar. A diferencia de las clases abstractas, una interfaz no puede contener estado (propiedades con valores iniciales) y una clase puede implementar múltiples interfaces, lo que permite la herencia múltiple.

Sintaxis:

```
interface Interfaz {
```



```
fun metodoDeInterfaz() // Método abstracto de la interfaz
}
```

Las interfaces pueden contener métodos con implementación utilizando `default` en Kotlin, aunque las clases que implementan una interfaz siempre deben proporcionar implementación para los métodos no implementados.

Ejemplo de interfaz:

```
interface Volador {
    fun volar() // Método sin implementación
}

class Pajaro : Volador {
    override fun volar() {
        println("El pájaro está volando")
    }
}

fun main() {
    val pajaro = Pajaro()
    pajaro.volar() // Salida: El pájaro está volando
}
```

Ejemplos

Ejemplo de clase simple con propiedades y métodos:

```
class Persona(val nombre: String, var edad: Int) {
    fun saludar() {
        println("Hola, me llamo $nombre y tengo $edad años.")
    }
}

fun main() {
    val personal = Persona("Juan", 25)
    personal.saludar() // Salida: Hola, me llamo Juan y tengo 25 años.
}
```

Ejemplo de herencia con una clase Vehículo

```
open class Vehiculo(val marca: String) {
    open fun encender() {
        println("El vehículo $marca está encendido.")
    }
}

class Moto(marca: String) : Vehiculo(marca) {
    override fun encender() {
        println("La moto $marca está encendida.")
    }
}
```



```
    }  
}  
  
fun main() {  
    val miMoto = Moto("Yamaha")  
    miMoto.encender() // Salida: La moto Yamaha está encendida.  
}
```

Ejemplo de implementación de una interfaz

```
interface Corredor {  
    fun correr()  
}  
  
class Atleta : Corredor {  
    override fun correr() {  
        println("El atleta está corriendo.")  
    }  
}  
  
fun main() {  
    val atleta = Atleta()  
    atleta.correr() // Salida: El atleta está corriendo.  
}
```

Referencias

Clases y herencia en Kotlin | *Android Developers*. (2024). Android Developers. <https://developer.android.com/codelabs/basic-android-kotlin-training-classes-and-inheritance?hl=es-419#0>

Chike Mgbemena. (2017, October 18). *Kotlin Desde Cero: clases abstractas, interfaces, herencia y alias Type*. Code Envato Tuts+; Envato Tuts. <https://code.tutsplus.com/es/kotlin-from-scratch-abstract-classes-interfaces-inheritance-and-type-alias--cms-29744t>

Reddit - Explora lo que quieras. (2024). Reddit.com. https://www.reddit.com/r/programacion/comments/10z0r9u/clases_abstractas_e_interfaces_etc_kotlin/